



## INTRODUCCIÓN A LA PROGRAMACIÓN ORIENTADA A OBJETOS

### Encapsulamiento y GUI

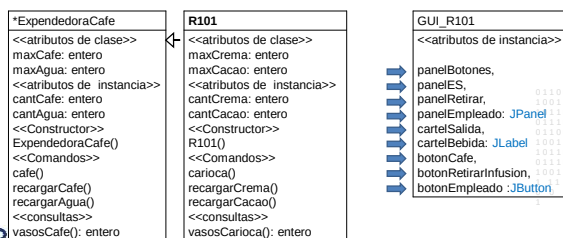
Depto. de Ciencias e Ingeniería de la Computación  
Universidad Nacional del Sur, Bahía Blanca

#### MÁQUINAS EXPENDEDORAS

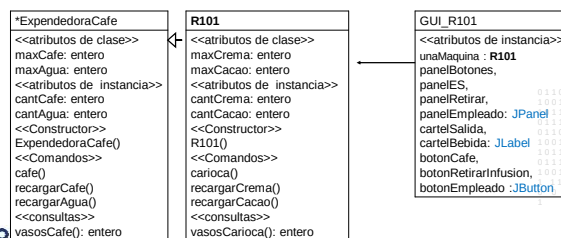
- Una fábrica produce tres tipos diferentes de máquinas expendedoras de café, **M111**, **R101** y **R101Plus**.
- Las máquinas de tipo **R101** preparan **café**, y **café carioca**.
- Tienen depósitos para **agua**, **café**, **crema** y **cacao**.
- Se desea implementar una interfaz gráfica para la máquina **R101** con botones para que:
  - el cliente pueda solicitar una bebida y
  - un botón para que el empleado recargue los depósitos.



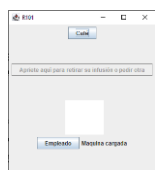
#### MÁQUINAS EXPENDEDORAS



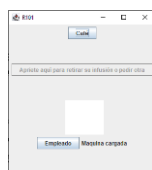
#### MÁQUINAS EXPENDEDORAS



#### MÁQUINAS EXPENDEDORAS



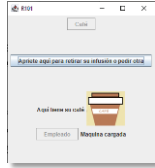
#### MÁQUINAS EXPENDEDORAS



- Inicialmente está habilitado el botón que le permiten al cliente obtener un café y el botón que le permite al empleado recargar los depósitos.

```
01100
10011
10110
01110
01100
10011
10011
01110
1001
5 11
0 0
1
```

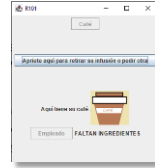
## MÁQUINAS EXPENDEDORAS



- Cuando el cliente oprime el botón **Café**, se sirve la infusión, se muestra un cartel en una etiqueta y se activa el botón que permite retirar el vaso.
- Cuando el cliente oprime este botón, se reactiva el botón que permite solicitar café y el botón para el empleado.

```
01100
10011
10110
01110
01100
10010
10110
10110
10110
10011
1 1 1
0 0
1
```

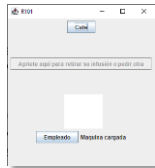
## MÁQUINAS EXPENDEDORAS



- Cuando la máquina queda sin ingredientes, la interface muestra un cartel que indica que **"FALTAN INGREDIENTES"**

```
01100
10011
10110
01110
01100
10010
10110
10110
10110
10011
10011
1 1 1
0 0
1
```

## MÁQUINAS EXPENDEDORAS



- Cuando el empleado oprime el botón **Empleado**, la interface vuelve a mostrar el cartel que indica **"Maquina cargada"**

```
01100
10011
10110
01110
01100
10010
10110
10110
10110
10011
1 1 1
0 0
1
```

```
//importar paquetes
class GUI_R101 extends JFrame {
    //Atributo de instancia para la clase asociada
    //Atributos de instancia para las componentes gráficas
    public GUI_R101 {
        //Establecer los atributos del frame
        //Crear y diagramar el panel para el cliente
        //Crear y diagramar el panel para los carteles
        //Crear y diagramar el panel para el empleado
        //Armar paneles
    }
    //Clases de los oyentes
}
```

```
//importar paquetes
import java.awt.*;
import javax.swing.*;
import java.awt.event.*;
import javax.swing.event.*;
class GUI_R101 extends JFrame {
    //Atributo de instancia para la clase asociada
    //Atributos de instancia para las componentes gráficas
    public GUI_R101 {
        //Establecer los atributos del frame
        //Crear y diagramar el panel para el cliente
        //Crear y diagramar el panel para los carteles
        //Crear y diagramar el panel para el empleado
        //Armar paneles
    }
    //Clases de los oyentes
}
```

```
//importar paquetes
import java.awt.*;
import javax.swing.*;
import java.awt.event.*;
import javax.swing.event.*;
class GUI_R101 extends JFrame {
    //Atributo de instancia para la clase asociada
    private R101 unaMaquina;
    //Atributos de instancia para las componentes gráficas
    private Container contenedor;
    private JPanel panelBotones, panelES, panelRetirar, panelEmpleado;
    private JLabel cartelSalida;
    private JLabel cartelBebida;
    private JLabel cartelEmpleado;
    private JButton botonCafe;
    private JButton botonRetirarInfusion;
    private JButton botonRetirarVaso;
}
```

La clase GUI\_R101 está asociada a la clase R101

```
...
public GUI_R101(String name){
    super(name);
    unaMaquina = new R101();
    //Captura el panel de contenido y establece su layout
    contenedor = getContentPane();
    contenedor.setLayout(new GridLayout(4,1));

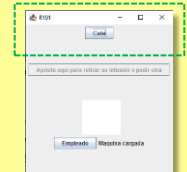
    armaPanelCliente();
    armaPanelCarteles();
    armaPanelEmpleado();
}
...
```

- Modulamos la solución para favorecer la legibilidad.

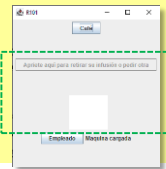
```
...
private void armaPanelCliente(){
    //Crea el panel y establece su apariencia
    panelBotones = new JPanel();
    panelBotones.setSize(369, 250);
    //Crea el boton de cafe, establece su apariencia, crea el oyente y lo
    registra
    botonCafe = new JButton();
    botonCafe.setText("Café");

    OyenteCafe oCafe = new OyenteCafe();
    botonCafe.addActionListener(oCafe);

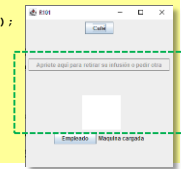
    panelBotones.add(botonCafe);
    contenedor.add(panelBotones);
}
...
```



```
private void armaPanelCarteles(){
    //Crea los paneles y establece su apariencia
    panelES = new JPanel();
    panelES.setSize(369, 51);
    panelRetirar = new JPanel();
    //Crea las etiquetas y establece su apariencia
    cartelSalida = new JLabel();
    cartelSalida.setLayout(new FlowLayout());
    cartelBebida = new JLabel();
    cartelBebida.setIcon(new ImageIcon("vacio.gif"));
}
...
```



```
...
private void armaPanelCarteles(){
    //Crea el botón y su oyente
    botonRetirarInfusion = new JButton();
    botonRetirarInfusion.setText("Apriete aqui para retirar su infusión o pedir otra");
    botonRetirarInfusion.setEnabled(false);
    OyenteRetirar oRetirar = new OyenteRetirar();
    botonRetirarInfusion.addActionListener(oRetirar);
    panelES.add(cartelSalida);
    panelES.add(cartelBebida);
    panelRetirar.add(botonRetirarInfusion);
    contenedor.add(panelRetirar);
    contenedor.add(panelES);
}
...
```



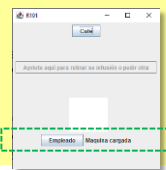
```
...
private void armaPanelEmpleado(){
    panelEmpleado = new JPanel();

    botonEmpleado = new JButton();
    botonEmpleado.setText("Empleado");

    OyenteEmpleado oEmpleado = new OyenteEmpleado();
    botonEmpleado.addActionListener(oEmpleado);

    cartelEmpleado = new JLabel("Maquina cargada");

    panelEmpleado.add(botonEmpleado);
    panelEmpleado.add(cartelEmpleado);
    contenedor.add(panelEmpleado);
}
...
```



```
...
private void deshabilitarBotones(){
    botonCafe.setEnabled(false);
    botonRetirarInfusion.setEnabled(true);
    botonEmpleado.setEnabled(false);
}

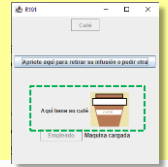
private void habilitarBotones(){
    botonCafe.setEnabled(true);
    botonRetirarInfusion.setEnabled(false);
    botonEmpleado.setEnabled(true);
}
...
```

# Introducción a la Programación Orientada a Objetos

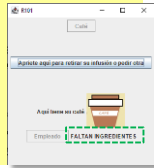
```
...
class OyenteCafe implements ActionListener{
    public void actionPerformed(ActionEvent evt) {

    }
}
...
```

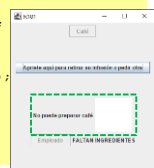
```
...
class OyenteCafe implements ActionListener{
    public void actionPerformed(ActionEvent evt) {
        //Solo prepara el cafe si la cantidad de ingredientes lo permite
        if (unaMaquina.vasosCafe()>=1){
            unaMaquina.cafe();
            cartelSalida.setText("Aquí tiene su café");
            cartelBebida.setIcon(new ImageIcon("Cafe.gif"));
        }
    }
}
...
```



```
...
class OyenteCafe implements ActionListener{
    public void actionPerformed(ActionEvent evt) {
        //Solo prepara el cafe si la cantidad de ingredientes lo permite
        if (unaMaquina.vasosCafe()>=1){
            unaMaquina.cafe();
            cartelSalida.setText("Aquí tiene su café");
            cartelBebida.setIcon(new ImageIcon("Cafe.gif"));
            if (unaMaquina.vasosCafe()==0)
                cartelEmpleado.setText("FALTAN INGREDIENTES");
        }
    }
}
...
```



```
...
class OyenteCafe implements ActionListener{
    public void actionPerformed(ActionEvent evt) {
        //Solo prepara el cafe si la cantidad de ingredientes lo permite
        if (unaMaquina.vasosCafe()>=1){
            unaMaquina.cafe();
            cartelSalida.setText("Aquí tiene su café");
            cartelBebida.setIcon(new ImageIcon("Cafe.gif"));
            if (unaMaquina.vasosCafe()==0)
                cartelEmpleado.setText("FALTAN INGREDIENTES");
        }
        else {
            cartelSalida.setText("No puede preparar café");
            cartelBebida.setIcon(new ImageIcon("vacio.gif"));
        }
        deshabilitarBotones();
    }
}
...
```



```
...
class OyenteRetirar implements ActionListener {
    public void actionPerformed(ActionEvent evt) {
        cartelSalida.setText("");
        cartelBebida.setIcon(new ImageIcon("vacio.gif"));
        habilitarBotones();
    }
}
...
```

```
...
class OyenteEmpleado implements ActionListener{
    public void actionPerformed(ActionEvent evt) {
        unaMaquina.recargarCafe();
        unaMaquina.recargarAgua();
        unaMaquina.recargarCrema();
        unaMaquina.recargarCacao();
        cartelEmpleado.setText("Maquina cargada");
    }
}
...
```

